第 8 章：面向对象编程

一句话先抓住本章：**Python 的类仍然负责把状态、规则和行为组织在一起，但它依靠公开属性、运行时协议和命名约定协作，不是把 Java 的字段、访问器和接口逐行换一种语法。**

本章面向已经熟悉 Java OOP、但刚开始学习 Python 的开发者。读完后，你应该能够：

- 判断一段逻辑该写成普通函数、类、继承关系还是组合关系；
- 解释类对象、实例、 `self` 和绑定方法在运行时是什么关系；
- 分清实例属性、类属性和 `property` ，知道一次读写最终落在哪里；
- 用 `__str__` 、 `__repr__` 、 `__eq__` 等协议接入 Python 内置行为；
- 正确选择实例方法、类方法和静态方法；
- 理解 `super()` 、MRO、鸭子类型以及 Python 的封装约定；
- 避开可变类属性、把类型提示当校验、机械生成 getter/setter 等迁移陷阱。

本章配套材料是 `example.py` 和 `Chapter08Oop.java` 。正文可以独立阅读，不需要先打开示例代码。

8.1 先完成 Java → Python 心智切换

两种语言都用类描述对象，但默认设计习惯不同：

要解决的问题	Java 常见做法	Python 常见做法与边界
创建对象	`new Account(...)` 调用构造器	直接调用类 `Account(...)` ； `__new__` 创建， `__init__` 初始化
表示当前对象	隐式的 `this`	定义方法时显式写 `self` ，调用绑定方法时自动传入
暴露状态	私有字段配 getter/setter	简单状态先用公开属性，需要规则时再升级为 `property`
限制访问	`private` / `protected` 由语言检查	`_name` 是内部约定； `__name` 主要避免子类意外重名，不是安全边界
提供多种构造方式	重载构造器或静态工厂	命名明确的 `@classmethod` 替代构造器
重载同名方法	按参数签名选择版本	后定义的同名方法覆盖前一个；通常改用默认参数或不同方法名

长... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代...

表达多态	继承和显式接口	既支持继承，也常按对象实际提供的协议进行鸭子类型协作
定义通用行为	覆写 `toString`、`equals` 等	实现 `__str__`、`__eq__` 等双下方法，接入语言协议

可以把 Python 的 `property` 暂时类比为 getter/setter，把 `classmethod` 类比为静态工厂，把鸭子类型类比为面向接口编程。但这些类比都不能完全等同：Python 的属性访问、方法绑定和类型约束发生在运行时，是否继承某个声明也往往不是协作的必要条件。

8.2 什么时候值得创建一个类

类维护的是一组长期共同变化的状态和规则

账户对象需要同时维护户主、余额、“余额不能为负”的约束以及存取款行为。这些内容共享同一生命周期，放进一个类能够形成清楚的业务边界：

```
class Account:
    def __init__(self, owner: str, balance: int = 0) -> None:
        if balance < 0:
            raise ValueError("余额不能为负")
        self.owner = owner
        self.balance = balance

    def deposit(self, amount: int) -> None:
        self.balance += amount
```

如果逻辑只是接收输入、完成计算、返回结果，而且不需要维护持续状态，普通函数通常更直接。Python 支持面向对象，但不要求把所有函数都包装进类。选择类的理由应是封装状态和不变量，而不是“Java 项目里通常有一个同名类”。

类本身也是运行时对象

Python 执行 `class Account: ...` 时，会执行类体并创建一个类对象，然后把名字 `Account` 绑定到它。调用这个类对象会创建实例：

```
account = Account("Alice", 10_000)

print(type(Account))  # <class 'type'>
print(type(account))  # <class '__main__.Account'> (模块名可能不同)
```

s/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-507f4a7da71874a9e2e41f14c6.html

的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代...

这里没有 `new`。`Account` 是类对象，`account` 是指向实例对象的名字。普通赋值只增加一个指向同一对象的名字，不会复制实例：

```
first = Account("Alice", 100)
second = first
second.deposit(50)

print(first.balance)    # 150
print(first is second)  # True
```

这与列表、字典等可变对象的引用语义相同，并不是 OOP 的特殊规则。

8.3 对象怎样创建，`self` 又从哪里来

`__new__` 创建实例，`__init__` 建立初始状态

调用 `Account("Alice")` 时，可以用下面的简化流程理解：

1. Python 调用 `Account.__new__` 创建实例；
2. 新实例作为 `self` 传给 `Account.__init__`；
3. `__init__` 在实例上建立完整、合法的初始状态；
4. 类调用表达式返回创建好的实例。

因此，`__init__` 严格说是初始化器，不负责分配对象，也必须返回 `None`。下面的类能被定义，但实例化时会失败：

```
class Wrong:
    def __init__(self) -> None:
        return 1  # TypeError: __init__() should return None
```

普通业务类几乎不需要重写 `__new__`。它主要出现在不可变类型子类、实例缓存或元编程中。初学 OOP 时，更重要的是让 `__init__` 一次建立好合法对象，而不是先返回半成品，再要求调用者按顺序调用多个 setter。

`self` 是显式写出的接收者

`self` 不是关键字，却是 Python 代码共同遵守的固定命名约定。定义实例方法时必须显式声明它：

...7f4a7da71874a9e2e41f14c6.html
...关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代...

```
class Account:
    def deposit(self, amount: int) -> None:
        self.balance += amount
```

调用时不需要手动传 `self` 。下面两种写法的核心效果相同：

```
account.deposit(100)          # 常规写法
Account.deposit(account, 100) # 直接从类取函数，并显式传实例
```

读取 `account.deposit` 时，Python 会得到一个**绑定方法**：它把函数和 `account` 组合在一起，已经记住了之后要使用的 `self` 。

```
deposit_later = account.deposit
print(deposit_later.__self__ is account)  # True
deposit_later(100)                        # 只需再传 amount
```

这也解释了为什么方法可以像普通函数一样保存到变量、放进容器或作为回调传递。

8.4 对象的状态到底存在哪里

实例属性保存独立状态，类属性保存共享默认值

在 `self` 上赋值通常创建实例属性；在类体中赋值创建类属性：

```
class Account:
    bank_name = "示例银行"  # 类属性：类创建时只执行一次

    def __init__(self, owner: str) -> None:
        self.owner = owner   # 实例属性：每次初始化一个实例时赋值
```

每个账户都有自己的 `owner` 。实例自身没有 `bank_name` 时，会从类及其父类读取该名称。给单个实例同名赋值，只会在该实例上创建一个遮蔽值：

```
alice = Account("Alice")
bob = Account("Bob")

alice.bank_name = "城东分行"
```

```
alice = Account("Alice")
bob = Account("Bob")

alice.bank_name = "城东分行"
print(alice.bank_name)    # 城东分行：来自 alice 实例
print(bob.bank_name)      # 示例银行：来自 Account 类
print(Account.bank_name)  # 示例银行：类属性没有改变
```

若要修改类级默认值，应显式写 `Account.bank_name = "新名称"` 。已经拥有同名实例属性的 `alice` 仍会继续读到自己的遮蔽值。

属性查找不只是“先实例、后类”

对 `obj.name` ，一个足够准确的入门模型是：

1. 先检查类及 MRO 上是否存在接管读写的**数据描述符**， `property` （包括只读 property）就属于它；
2. 如果没有，再检查实例自己的属性字典；
3. 再检查类及其父类上的普通属性或方法；
4. 仍找不到时，通常抛出 `AttributeError` 。

这个顺序解释了两个看似不同的现象：普通类属性可以被实例属性遮蔽，而 `property` 能优先拦截同名属性的赋值。描述符是 Python 对象模型的通用机制；本章只需要掌握 `property` 带来的实际读写流程，更完整的数据模型会在后续章节继续展开。

最危险的类属性是可变容器

类体只执行一次，因此类属性中的列表也只创建一次：

```
class BadShoppingCart:
    items: list[str] = []


first = BadShoppingCart()
second = BadShoppingCart()
first.items.append("Python 书")
print(second.items)  # ['Python 书']：两个实例读到同一个列表
```

每个实例需要独立容器时，必须在 `__init__` 中创建：

```
class ShoppingCart:
    def __init__(self) -> None:
        self.items: list[str] = []
```

它与“可变默认参数只创建一次”的陷阱根因相同：对象是在定义时创建一次，还是在每次初始化时重新创建，决定了状态是否共享。

动态属性和类型提示都不会替你守住业务规则

普通 Python 实例通常允许初始化后新增属性，例如 `account.note = "VIP"` 。这种能力适合探索，但稳定的业务对象仍应在 `__init__` 中统一建立主要状态，否则维护者、IDE 和类型检查器都很难判断实例有哪些属性。拼写错误也可能悄悄创建一个错误的新属性。

同样， `balance: int` 是给阅读者、IDE 和静态类型检查器看的提示，Python 默认不会据此插入运行时校验或转换：

```
class Account:
    def __init__(self, balance: int) -> None:
        self.balance = balance


account = Account("100")  # 运行时可能接受，balance 实际是 str
```

余额非负、订单状态合法等不变量必须由业务代码显式检查。配套示例还特意拒绝 `bool` ，因为 Python 中 `bool` 是 `int` 的子类： `isinstance(True, int)` 会得到 `True` 。若金额必须是纯整数，可以用 `type(value) is int` 明确排除布尔值。

8.5 用 `property` 维护属性不变量

先保持自然的访问语法，再在边界集中校验

Java 常从私有字段和 getter/setter 起步。Python 更倾向先公开简单属性；当属性需要校验、计算、只读控制或 API 兼容时，再把它升级为 `property` ，调用方仍然使用自然的属性语法：

```
class Account:
    def __init__(self, balance: int = 0) -> None:
        self.balance = balance
```

```python
class Account:
    def __init__(self, balance: int = 0) -> None:
        self.balance = balance

    @property
    def balance(self) -> int:
        return self._balance

    @balance.setter
    def balance(self, value: int) -> None:
        if type(value) is not int:
            raise TypeError("余额必须是整数")
        if value < 0:'
            raise ValueError("余额不能为负")
        self._balance = value
```

这里没有保存两份余额：

- `balance` 是类上的 `property` 对象，是公开读写入口；
- `_balance` 是实例上的普通属性，才是实际存储位置；
- `self.balance = balance` 会调用 setter，校验通过后才创建 `self._balance`。

`@property` 先把 getter 包装成名为 `balance` 的 property。随后 `@balance.setter` 在这个 property 上注册写入函数。因此不能写 `@_balance.setter`：`_balance` 只是实例运行时才创建的存储属性，并不是类上的 property。

一次读取和一次赋值怎样流动

读取 `account.balance` 时：

1. Python 在 `Account` 上找到 `balance` property；
2. property 调用 getter，并把 `account` 作为 `self`；
3. getter 返回 `account._balance`。

执行 `account.balance = 100` 时：

1. Python 找到同一个 property；
2. property 调用 setter，传入 `account` 和 `100`；
3. setter 先验证类型和范围；
4. 验证通过后，把 `100` 写入 `account._balance`。

setter 内必须写 `self._balance = value`。若再次写 `self.balance = value`，赋值会重新进入同一个 setter，最终因无限递归抛出 `RecursionError`。

`self.balance += amount` 不是绕过 property 的特殊写法，可以展开理解为：

```
new_balance = self.balance + amount  # getter 读取 _balance
self.balance = new_balance           # setter 校验并写回 _balance
```

初始化、外部赋值、存款和利息等路径都应该维护同一套不变量。它们可以统一经过 setter，也可以复用内部校验函数；关键是不能让某条写入路径成为绕过规则的后门。

只读属性与不适合 property 的操作

只定义 getter、不定义 setter，就能得到公开只读属性：

```
class Wallet:
    def __init__(self, balance: int) -> None:
        self._balance = balance

    @property
    def balance(self) -> int:
        return self._balance
```

读取 `wallet.balance` 正常，执行 `wallet.balance = 100` 会抛出 `AttributeError`。但这仍不是安全隔离：外部技术上可以修改 `_balance`，只是明确违反了类的公开契约。

property 适合表现“读取起来像数据”的轻量行为。若操作昂贵、包含网络或磁盘 I/O，或者调用者需要意识到它会改变系统状态，显式方法如 `refresh_balance()`、`load_profile()` 更清楚。

8.6 三种方法分别绑定什么

判断方法种类时，不要先问“Java 中是不是 static”，先问实现需要哪一个隐式对象：

写法	自动绑定的首个参数	典型用途
实例方法	`self`，当前实例	读取或修改实例状态
`@classmethod`	`cls`，实际调用该方法的类	可继承的替代构造器、类级行为

README × README
vf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-507f4a7da71874a9e2e41f14c6.html
目关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下

写法	自动绑定的首个参数	典型用途
实例方法	`self`，当前实例	读取或修改实例状态
`@classmethod`	`cls`，实际调用该方法的类	可继承的替代构造器、类级行为
`@staticmethod`	无	与类概念紧密相关，但不需要实例或类状态的规则

类方法适合命名明确的替代构造器

Python 不按参数签名重载构造器，常用类方法表达“从文本创建”“从配置创建”等输入来源：

```
from typing import Self


class Account:
    def __init__(self, owner: str, balance: int = 0) -> None:
        self.owner = owner
        self.balance = balance

    @classmethod
    def from_text(cls, text: str) -> Self:
        owner, balance = text.split(":", maxsplit=1)
        return cls(owner.strip(), int(balance))
```

执行 `Account.from_text("Alice:100")` 时，`cls` 是 `Account`。若子类继承该方法并调用 `SavingsAccount.from_text(...)`，`cls` 就是 `SavingsAccount`。因此替代构造器应调用 `cls(...)`，而不是写死 `Account(...)`；前提是子类构造签名仍兼容这里传入的参数。

`Self` 是类型提示，表示返回值与实际调用方法的类一致，不会改变运行时行为。可以先把类方法类比为 Java 的命名静态工厂，但不能完全等同：类方法自动接收实际调用的类，并能保留可继承性。

静态方法只是没有自动绑定参数的函数

```
class Account:
    @staticmethod
    def is_valid_amount(amount: object) -> bool:
        return type(amount) is int and amount > 0
```

该规则与账户概念紧密相关，却不读取某个账户或类状态，所以可以放在 `Account` 中作为静态方法。

…查规则与 `Account` 的关系并不紧密，模块级普通函数通常更自然。

重复定义同名方法不会形成重载

```
class Parser:
    def parse(self, text: str) -> str:
        return text

    def parse(self, number: int) -> str:
        return str(number)  # 直接覆盖前一个 parse
```

Python 通常用默认参数、仅关键字参数、不同且有业务含义的方法名，或者 `from_text` 这样的替代构造器表达多种调用方式。确实需要按类型分派时，再考虑 `functools.singledispatchmethod`。

8.7 用双下方法接入 Python 协议

`__str__`、`__repr__`、`__eq__`、`__len__` 等特殊方法规定对象如何参与通用语言操作。业务代码应写 `str(obj)`、`left == right`、`len(obj)`，而不是直接调用双下方法。

用户代码	Python 使用的协议	设计目标
`print(obj)` / `str(obj)`	`__str__`	面向最终用户的可读文本
`repr(obj)` / 调试容器	`__repr__`	面向开发者的明确表示
`left == right`	`__eq__`	领域意义上的值相等
`len(obj)`	`__len__`	对象包含的元素数量

这些名字由 Python 数据模型保留。不要自创 `__save_data__` 一类双下名字；普通业务方法应该使用正常名称。

`str` 与 `repr` 面向不同读者

```
class Account:
    def __str__(self) -> str:
        return f"{self.owner}: {self.balance} 分"
```

README
README
f7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-507f4a7da71874a9e2e41f14c6.html
关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单

`str` 与 `repr` 面向不同读者

```
class Account:
    def __str__(self) -> str:
        return f"{self.owner}: {self.balance} 分"

    def __repr__(self) -> str:
        return f"Account(owner={self.owner!r}, balance={self.balance})"
```

`str` 优先让用户读懂，类似 Java 中面向展示的格式化结果。`repr` 优先帮助开发者定位对象，理想情况下接近一个可重建对象的表达式。`!r` 会调用字段自身的 `repr`，因此字符串的引号和转义不会丢失。密码、令牌等敏感字段必须隐藏，不能为了“完整调试信息”而泄露秘密。

`==` 比较领域值，`is` 比较对象身份

```
class Account:
    def __eq__(self, other: object) -> bool:
        if not isinstance(other, Account):
            return NotImplemented
        return (self.owner, self.balance) == (other.owner, other.balance)
```

- `first is second` 判断两个名字是否指向同一个对象，不能重载；
- `first == second` 询问两个对象在领域中是否值相等，可以由 `__eq__` 定义；
- 不知道如何比较某种类型时返回 `NotImplemented`，让 Python 尝试另一侧的实现；
- `NotImplemented` 是一个特殊值，不是表示“抽象方法未实现”的 `NotImplementedError` 异常。

相等规则还会影响哈希。若两个对象 `==`，它们的哈希值也必须相同。账户余额会变化，把这种对象作为 `set` 元素或字典键很危险；Python 在类只定义 `__eq__` 而没有合理的 `__hash__` 时，通常会让实例不可哈希。不要随手补一个与相等规则不一致的哈希实现。

8.8 继承复用契约，组合拆分职责

`super()` 按 MRO 找下一个实现

`SavingsAccount` 是一种 `Account`，子类先复用账户初始化规则，再创建自己的利率状态：

```
class SavingsAccount(Account):
```

`super()` 按 MRO 找下一个实现

`SavingsAccount` 是一种 `Account`，子类先复用账户初始化规则，再创建自己的利率状态：

```
class SavingsAccount(Account):
    def __init__(
        self,
        owner: str,
        balance: int = 0,
        interest_rate: float = 0.02,
    ) -> None:
        super().__init__(owner, balance)
        self.interest_rate = interest_rate
```

`super()` 不能简单理解为“固定调用直接父类”。它会按照方法解析顺序（MRO）找到当前类之后的下一个实现；`SavingsAccount.mro()` 可以查看具体顺序。这个机制使协作式多继承成为可能，也要求参与协作的方法使用兼容的调用约定。

Python 语法不强制 `super().__init__()` 是第一条语句，但通常应先让父类建立完整、合法的基础状态。子类后续逻辑就可以安全依赖这些状态；父类初始化失败时，也不会先执行无意义的子类修改。

覆写方法不仅是复用实现，还要遵守父类公开契约。任何能接收 `Account` 的代码拿到 `SavingsAccount` 后，仍应能按相同语义存款、取款和读取余额。这就是替换关系，而不只是语法上写了一个父类名。

Python 也常按行为完成多态

继承自 `Account` 的对象可以放入 `list[Account]`，由同一段代码调用共同方法。除此之外，Python 还大量采用鸭子类型：调用者关注对象能不能完成需要的行为，不一定要求它继承某个共同父类。

可以先把它类比为 Java 的面向接口编程，但不能完全等同。Java 通常要求显式 `implements`；Python 运行时往往直接尝试调用所需方法。大型项目可以用 `typing.Protocol` 把这种结构契约告诉静态类型检查器，协议的系统用法会在后续章节展开。

“拥有一个”通常应该使用组合

银行卡不是一种账户，而是拥有并使用一个账户。用组合可以把支付限额和余额规则放在不同职责中：

```
class BankCard:
```

“拥有一个”通常应该使用组合

银行卡不是一种账户，而是拥有并使用一个账户。用组合可以把支付限额和余额规则放在不同职责中：

```
class BankCard:
    def __init__(self, account: Account, limit: int) -> None:
        self.account = account
        self.limit = limit

    def pay(self, amount: int) -> None:
        if amount > self.limit:
            raise ValueError("超过银行卡单笔限额")
        self.account.withdraw(amount)
```

`BankCard` 保存的是传入账户的引用，并没有复制余额。`pay()` 把扣款委托给账户，因此账户继续负责“金额有效、余额不能透支”等规则，银行卡只维护自己的单笔限额。

选择关系时可以先问两个问题：

- 子类能否在所有需要父类的位置保持父类契约？能，才可能是“是一种”的继承；
- 新对象只是需要另一个对象提供服务吗？是，通常应使用“拥有一个”的组合。

不要为了复用几行代码建立不自然的父子关系。多继承和复杂 MRO 是 Python 提供的能力，并不是默认设计目标。

8.9 Python 的封装依靠公开契约

单下划线表示内部实现，不阻止访问

`_balance` 表示“这是内部细节，外部调用者不要依赖”。类本身和受信任的子类仍可以使用它，IDE、自动补全和文档工具也会把它视为非公开名称。但 Python 不会因此阻止访问：

```
account._balance = -1  # 技术上可能成功，但主动绕过了 Account 的公开契约
```

Python 的封装主要依靠清楚的文档、稳定的公开接口、property 或业务方法维护不变量，以及测试保护公开行为。“技术上能访问”不等于“调用方可以依赖”。

双下划线主要用于避免子类意外重名

双下划线主要用于避免子类意外重名

类体中的 `self.__state` 会触发名称改写（name mangling），大致改为 `self._ClassName__state`：

```
class Base:
    def __init__(self) -> None:
        self.__state = "base"
```

它的主要作用是避免子类无意间定义同名属性，不是制造真正的 Java `private` 字段。知道改写后名称的调用者仍可能访问它，所以更不能用它保护密钥或令牌。普通内部状态优先使用单下划线；只有设计可继承的基类、确实要避免子类命名冲突时，才考虑双下划线。

Python 也没有 Java 式 `protected`。设计给别人继承的类时，应明确哪些方法是稳定扩展点，避免子类依赖偶然的内部布局。配套代码中的 `SavingsAccount.add_interest()` 通过公开的 `balance` property 更新余额，而不是直接写父类 `_balance`，因此父类以后加强校验时，子类仍会经过规则。

8.10 Java 工程师最容易踩的坑

错误判断	失败表现	更合适的做法
把每个实例的 `items = []` 写在类体中	不同实例意外共享同一个列表	在 `__init__` 中创建 `self.items`
认为 `balance: int` 会拦截字符串	错误类型进入对象，稍后才在运算中失败	对业务不变量做显式运行时校验
为每个字段预先生成 getter/setter	类充满没有规则的样板访问器	先用公开属性，需要规则时升级为 property
在 property setter 中给同一公开属性赋值	setter 无限递归，最终 `RecursionError`	校验后写内部存储属性，如 `_balance`
重复定义同名方法模拟 Java 重载	前面的定义被后面的覆盖	默认参数、关键字参数、不同方法名或类方法
用 `==` 判断是不是同一个实例	值相等的两个对象被误判成同一对象	身份比较用 `is`，领域值比较用 `==`
写死 `Account(...)` 实现类方法构造器	子类调用后仍得到父类实例	使用自动绑定的 `cls(...)`

2502x6dg8fpjj0sff40000gp/T/crossnote-f-507f4a7da71874a9e2e41f14c6.html
eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

写死 `Account(...)` 实现类方法构造器	子类调用后仍得到父类实例	使用自动绑定的 `cls(...)`
为复用代码强行继承	子类无法维持父类语义， 层级越来越脆弱	先验证 “is-a”，否则优先组合
用 `_name` 或 `__name` 保存秘密	调试、 序列化或主动访问仍可能暴露数据	敏感信息不进入公开表示， 并单独设计安全边界
`__eq__` 遇到陌生类型就抛异常	正常的跨类型比较意外中断	返回 `NotImplemented`

最后记住一个设计顺序：先找对象必须长期维护的状态和不变量，再确定公开协议，最后才选择普通属性、property、继承或组合。不要从“怎样把 Java 类翻译成更短的 Python”开始。

8.11 运行本章项目验证心智模型

本章只使用标准库，不需要安装第三方依赖。仓库配置要求 Python 3.14 或更高版本。请在包含 `pyproject.toml` 的**项目根目录**执行：

```
python3 --version
python3 java_to_python/chapters/08_oop/example.py
```

运行 Python 示例时，重点观察：

1. 绑定方法的 `__self__` 是否就是创建方法的账户实例；
2. 修改一个实例的 `bank_name` 后，类属性和另一个实例是否保持不变；
3. 错误购物车的列表为何共享，正确购物车的列表为何独立；
4. `vars(account)` 中为何有 `_balance`，却没有第二份 `balance`；
5. 负余额、字符串余额和布尔金额分别在哪一层被拒绝；
6. 两个账户为何可以满足 `first == second`，同时 `first is not second`；
7. `SavingsAccount.from_text(...)` 为何创建子类，而不是固定创建 `Account`；
8. 银行卡支付后，传入的原账户余额为何也发生变化。

也可以使用统一运行器同时对照 Python 与 Java 版本：

```
python3 java_to_python/run_all.py --chapter 8
```

eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

Java 对照文件展示传统私有字段、getter、静态工厂、 `equals` 、 `hashCode` 和继承写法。它用于帮助迁移心智模型，不表示两份实现的每个设计都严格一一对应。统一运行器会把 Java 编译产物放入临时目录，不会在源码目录留下 `.class` 文件。

本章总结

- 类适合共同维护状态、业务不变量和行为；无持续状态的计算通常优先用普通函数。
- 类本身是对象；调用类创建实例， `__init__` 初始化实例，普通赋值不会复制对象。
- `self` 是显式声明、调用绑定方法时自动传入的当前实例。
- 实例属性保存独立状态；类属性保存共享状态或默认值，可变类属性尤其危险。
- `property` 是公开访问入口，内部属性才保存数据；所有写入路径都应维护同一套不变量。
- `classmethod` 接收实际调用的类，适合可继承的替代构造器； `staticmethod` 不接收隐式对象。
- 双下方法让对象参与 Python 协议； `==` 是领域值相等， `is` 是对象身份。
- 继承表达可替换的"是一种"，组合表达"拥有一个"；复用几行代码本身不是继承理由。
- `_name` 和 `__name` 都不是访问控制或安全机制，稳定封装来自清楚的公开契约。

小练习

练习 1：基础对象与不变量

创建 `Wallet` 类：

- 初始化时接收所有者和以"分"为单位的余额；
- 余额必须是非负整数，且 `True` 不能作为金额；
- 余额只允许通过 `deposit()` 和 `withdraw()` 修改，金额必须为正且不能透支；
- `balance` 对调用方只读；
- 分别实现适合用户查看的 `__str__` 和适合调试的 `__repr__` 。

验证零余额、负数、字符串金额、刚好取完和透支五种边界。

练习 2：替代构造器与相等

为 `Wallet` 增加 `from_text("Alice:100")` 类方法和 `__eq__` ：

- 输入缺少冒号时抛出含义清楚的异常；
- 比较另一个 `Wallet` 时按所有者和余额判断；
- 比较不支持的类型时返回 `NotImplemented` ；

小练习

练习 1：基础对象与不变量

创建 `Wallet` 类：

- 初始化时接收所有者和以“分”为单位的余额；
- 余额必须是非负整数，且 `True` 不能作为金额；
- 余额只允许通过 `deposit()` 和 `withdraw()` 修改，金额必须为正且不能透支；
- `balance` 对调用方只读；
- 分别实现适合用户查看的 `__str__` 和适合调试的 `__repr__` 。

验证零余额、负数、字符串金额、刚好取完和透支五种边界。

练习 2：替代构造器与相等

为 `Wallet` 增加 `from_text("Alice:100")` 类方法和 `__eq__` ：

- 输入缺少冒号时抛出含义清楚的异常；
- 比较另一个 `Wallet` 时按所有者和余额判断；
- 比较不支持的类型时返回 `NotImplemented` ；
- 验证两个值相等的钱包为什么仍可能满足 `first is not second` 。

练习 3：比较继承与组合

先实现 `LimitedWallet` 子类，增加每日提现上限。再实现 `GiftCard` ，让它拥有一个 `Wallet` 并增加单笔消费上限。说明哪个关系是“是一种”，哪个关系是“拥有一个”，以及失败路径为什么不能提前修改余额或已用额度。

练习 4：定位共享状态 bug

故意把交易记录列表写成类属性，创建两个钱包历史对象，只向其中一个添加记录，观察另一个对象为何也出现记录。然后把列表移入 `__init__` 修复，并用 `is` 验证两个列表已经是不同对象。

配套 `example.py` 包含这些练习的可运行参考实现。建议先独立完成，再对照实现并修改余额、利率和限额，分别观察成功路径与异常路径。

```python
"""第 8 章：用可运行示例理解 Python 面向对象。

本章不只展示语法，还会打印对象状态、方法绑定和属性查找的结果。建议先猜每一段
输出，再运行文件验证自己的心智模型。

阅读本文件时先抓住四条主线：

1. ``self`` 是当前实例，``self.xxx`` 通常表示这个实例上的状态。
2. ``balance`` 和 ``_balance`` 不是两份余额：前者是公开 property，后者才是实际存储值。
3. ``@classmethod`` 自动接收“实际调用的类” ``cls``；``@staticmethod`` 不自动接收对象。
4. 子类先用 ``super().__init__`` 建好父类部分，再初始化自己新增的部分，通常最容易维护。
"""

from typing import ClassVar, Self


def show_section(title: str) -> None:
    """让每个知识点的终端输出拥有清晰边界。"""
    print(f"\n{'=' * 12} {title} {'=' * 12}")


class Account:
    """一个维护“余额不能为负”这一业务不变量的账户。

    对外公开 ``account.balance``，让调用者像访问普通字段一样读写余额；对内只用
    ``account._balance`` 保存真正的数值。property 位于二者中间，负责拦截公开读写并校验。

    可以把它想成：``balance`` 是带门卫的窗口，``_balance`` 是窗口后面的仓库。窗口和仓库
    指向的是同一份业务状态，并不是两个余额变量。
    """

    bank_name: str = "示例银行"  # 类属性：保存在类上，所有实例默认都能读到

    def __init__(self, owner: str, balance: int = 0) -> None:
        """初始化已经由 Python 创建好的实例。

        类型提示不会在运行时自动检查参数，所以重要的业务约束仍需显式校验。
```

```
22  class Account:
34      def __init__(self, owner: str, balance: int = 0) -> None:
37          类型提示不会在运行时自动检查参数，所以重要的业务约束仍需显式校验。
38          初始余额通过 property setter 写入，让初始化和后续赋值共用同一套余额规则。
39          """
40          if not isinstance(owner, str):
41              raise TypeError("户主必须是字符串")
42
43          normalized_owner: str = owner.strip()
44          if not normalized_owner:
45              raise ValueError("户主不能为空")
46
47          # owner 不需要拦截读写，所以直接保存为普通的公开实例属性。
48          self.owner: str = normalized_owner
49
50          # 这行看起来像普通赋值，实际会调用下面的 balance setter:
51          #
52          #     Account.balance.fset(self, balance)
53          #
54          # setter 校验成功后才创建 self._balance。实例中不会再额外保存一份名为
55          # self.balance 的数据，因此这里没有“两个余额”。
56          self.balance = balance
57
58      # 第一步：@property 把这个 getter 函数包装成类属性 Account.balance。
59      # 以后读取 account.balance 时，Python 会自动调用 balance(account)。
60      @property
61      def balance(self) -> int:
62          """读取公开属性 ``balance`` 时，返回内部真正保存的 ``_balance``。"""
63          return self._balance
64
65      # 第二步：此时 balance 已经是 property 对象；@balance.setter 的意思是
66      # “给这个名为 balance 的 property 注册写入函数”。
67      # 不能写 @_balance.setter，因为 _balance 只是实例运行时才出现的普通值，
68      # 类定义到这里时不存在名为 _balance 的 property 对象。
69      @balance.setter
70      def balance(self, value: int) -> None:
```

```python
class Account:
    @balance.setter
    def balance(self, value: int) -> None:
        """拦截 ``account.balance = value``，校验后写入唯一的存储字段。"""
        # bool 是 int 的子类，因此这里用 type(value) is int 明确拒绝 True/False。
        if type(value) is not int:
            raise TypeError("余额必须是整数（单位：分）")
        if value < 0:
            raise ValueError("余额不能为负")

        # _balance 是真正保存数值的实例属性。这里必须写 _balance，不能再写
        # self.balance = value，否则会再次进入当前 setter，造成无限递归。
        self._balance: int = value

    def deposit(self, amount: int) -> None:
        """向当前 ``self`` 账户存款，并让新余额继续经过 setter。"""
        if not self.is_valid_amount(amount):
            raise ValueError("存款金额必须为正整数")

        # self.balance += amount 可以展开理解成：
        #
        #     new_balance = self.balance + amount  # 先调用 getter
        #     self.balance = new_balance           # 再调用 setter
        #
        # 因而最终仍由 setter 把结果保存到 self._balance。
        self.balance += amount

    def withdraw(self, amount: int) -> None:
        """从当前账户取款；校验失败时不会改变余额。"""
        if not self.is_valid_amount(amount):
            raise ValueError("取款金额必须为正整数")
        if amount > self.balance:
            raise ValueError("余额不足")
        self.balance -= amount

    def __str__(self) -> str:
        """面向最终用户的可读文本，由 str() 和 print() 使用。"""
```

```python
class Account:
    def __str__(self) -> str:
        """面向最终用户的可读文本，由 str() 和 print() 使用。"""
        return f"{self.owner}: {self.balance} 分"

    def __repr__(self) -> str:
        """面向开发者的调试文本，尽量暴露重建对象所需的关键信息。"""
        return f"Account(owner={self.owner!r}, balance={self.balance})"

    def __eq__(self, other: object) -> bool:
        """定义领域中的“值相等”，而不是比较两个变量是否指向同一对象。"""
        if not isinstance(other, Account):
            # 让 Python 有机会尝试 other 的反向比较；双方都不支持时结果才是 False。
            return NotImplemented
        return (self.owner, self.balance) == (other.owner, other.balance)

    @classmethod
    def from_text(cls, text: str) -> Self:
        """把 ``"户主:余额"`` 文本转换成对象，是一个有名字的替代构造器。

        这不是反射。``Account`` 本来就是可以调用的类对象；@classmethod 只是在调用本方法时
        自动把类传给 ``cls``：

        - ``Account.from_text(...)`` 中，``cls is Account``；
        - ``SavingsAccount.from_text(...)`` 中，``cls is SavingsAccount``。

        Java 可以把它类比为 ``static Account fromText(String text)`` 工厂方法，但 Python 的
        ``cls(...)`` 会保留实际调用的子类。返回类型 ``Self`` 只给 IDE/类型检查器看。
        """
        # str.partition(":") 总是返回三部分：冒号左边、冒号本身、冒号右边。
        # 如果原文本没有冒号，separator 就是空字符串。
        owner: str, separator: str, balance_text: str = text.partition(":")
        if not separator:
            raise ValueError("账户文本必须使用“户主:余额”格式")

        # 调用一个类对象就会创建实例。这里等价于：
```

```python
class Account:
    def from_text(cls, text: str) -> Self:
        # 调用一个类对象就会创建实例。这里等价于：
        # Account.from_text 时调用 Account(owner, ...)，子类调用时则创建子类实例。
        return cls(owner, int(balance_text.strip()))

    @staticmethod
    def is_valid_amount(amount: object) -> bool:
        """检查金额；既不读取实例 ``self``，也不读取类 ``cls``。

        这与 Java 静态方法最接近。它放在 Account 中只是因为规则与账户概念紧密相关；如果很多
        不同类型都要使用这条规则，也可以把它写成模块级普通函数。
        """
        return type(amount) is int and amount > 0


class SavingsAccount(Account):
    """SavingsAccount 是一种 Account，继承户主、余额、存款和取款规则。"""

    def __init__(self, owner: str, balance: int = 0, interest_rate: float = 0.02) -> None:
        # super() 按 MRO 找下一个 __init__，在这里就是 Account.__init__。
        # 它先建立 owner 和 balance，保证父类部分已经是合法、完整的 Account。
        #
        # Python 语法并不强制 super() 必须是第一条语句；在当前简单实现中，先设置
        # interest_rate 也碰巧能工作。但通常应该先初始化父类，因为子类后续代码、property
        # 或父类方法可能依赖 owner/balance。父类初始化失败时，也能避免先留下子类的半成品状态。
        super().__init__(owner, balance)

        # 这会进入下面的 interest_rate setter；校验后由 _interest_rate 保存实际值。
        self.interest_rate = interest_rate

    # 与 balance/_balance 完全相同：interest_rate 是公开入口，_interest_rate 是内部存储。
    @property
    def interest_rate(self) -> float:
        """读取公开利率时，返回内部存储值。"""
        return self._interest_rate
```

```python
class SavingsAccount(Account):

    # 与 balance/_balance 完全相同：interest_rate 是公开入口，_interest_rate 是内部存储。
    @property
    def interest_rate(self) -> float:
        """读取公开利率时，返回内部存储值。"""
        return self._interest_rate

    # 必须写 @interest_rate.setter，因为 setter 要注册到名为 interest_rate 的 property 上。
    @interest_rate.setter
    def interest_rate(self, value: float) -> None:
        """拦截公开利率赋值，把合法的 int/float 统一保存成 float。"""
        if isinstance(value, bool) or not isinstance(value, (int, float)):
            raise TypeError("利率必须是数字")
        if not 0 <= value <= 1:
            raise ValueError("利率必须在 0 到 1 之间")

        # 不写 self.interest_rate = ...，否则会递归调用当前 setter。
        self._interest_rate = float(value)

    def add_interest(self) -> int:
        """用两个公开 property 计算利息，再通过余额 setter 写回。"""
        # self.balance 会调用父类的 balance getter；self.interest_rate 会调用当前类的
        # interest_rate getter。它们返回的实际值分别来自 _balance 和 _interest_rate。
        interest: int = round(self.balance * self.interest_rate)

        # 这行先通过 getter 读取旧余额，计算新值，然后通过 setter 写回 _balance。
        # 使用公开接口能继续复用 Account 对余额的约束，子类不需要知道父类怎样存储余额。
        self.balance += interest
        return interest

    def __repr__(self) -> str:
        return (
            f"SavingsAccount(owner={self.owner!r}, balance={self.balance}, "
            f"interest_rate={self.interest_rate})"
        )

```

```python

class BankCard:
    """银行卡"拥有"账户而不是"属于"账户，用组合比继承更贴合关系。"""

    def __init__(self, account: Account, single_payment_limit: int = 5_000) -> None:
        # 静态方法没有 self/cls，可以直接通过类名调用。
        if not Account.is_valid_amount(single_payment_limit):
            raise ValueError("单笔限额必须为正整数")

        # 保存的是 Account 对象的引用，不是复制一份账户。card.pay 最终会修改同一个账户。
        self.account: Account = account
        self.single_payment_limit: int = single_payment_limit

    def pay(self, amount: int) -> None:
        if not Account.is_valid_amount(amount):
            raise ValueError("消费金额必须为正整数")
        if amount > self.single_payment_limit:
            raise ValueError("超过银行卡单笔限额")
        # 把实际扣款委托给账户，继续复用 Account 的余额不足等规则。
        self.account.withdraw(amount)


class BadShoppingCart:
    """教学反例：可变类属性会被所有实例共享。"""

    # 类体只执行一次，因此这个列表也只创建一次，所有没遮蔽它的实例都会读到同一个列表。
    # ClassVar 只是在类型层面明确"这是类属性"，不会为每个实例复制列表。
    items: ClassVar[list[str]] = []


class ShoppingCart:
    """正确示例：需要各实例独立的数据应在 __init__ 中创建。"""

    def __init__(self) -> None:
        # 每次 ShoppingCart() 都会执行 __init__，所以每个实例得到一个新列表。
        self.items: list[str] = []

```

```python
class Wallet:
    """README 练习：只能通过公开行为修改余额的钱包。

    它与 Account 刻意采用不同设计：Wallet 的 balance 只有 getter，没有 setter，因此外部只能
    读取 ``wallet.balance``，不能写 ``wallet.balance = ...``。余额变化只能通过 deposit 和
    withdraw 这些有明确业务含义的方法完成。
    """

    def __init__(self, owner: str, balance: int = 0) -> None:
        """直接校验并建立钱包状态；这里没有可复用的 balance setter。"""
        if not isinstance(owner, str) or not owner.strip():
            raise ValueError("所有者必须是非空字符串")
        if type(balance) is not int:
            raise TypeError("余额必须是整数（单位：分）")
        if balance < 0:
            raise ValueError("余额不能为负")
        self.owner: str = owner.strip()

        # Wallet 的公开 balance 是只读 property，所以类自己的受信任方法直接维护 _balance。
        self._balance: int = balance

    # 只有 @property getter、没有 @balance.setter，便得到公开只读属性。
    @property
    def balance(self) -> int:
        """允许读取 ``wallet.balance``，但不提供 ``wallet.balance = value``。"""
        return self._balance

    @staticmethod
    def _validate_positive_amount(amount: object) -> int:
        """供 Wallet 及子类复用的内部金额校验工具。"""
        if type(amount) is not int or amount <= 0:
            raise ValueError("金额必须是正整数（单位：分）")
        return amount

    def deposit(self, amount: int) -> None:
        """校验后直接修改内部存储；外部调用者仍不能直接赋值 balance。"""
```

```python
class Wallet:

    def deposit(self, amount: int) -> None:
        """校验后直接修改内部存储；外部调用者仍不能直接赋值 balance。"""
        self._balance += self._validate_positive_amount(amount)

    def withdraw(self, amount: int) -> None:
        """先完成全部校验，最后才修改余额，避免失败路径留下半次修改。"""
        valid_amount: int = self._validate_positive_amount(amount)
        if valid_amount > self._balance:
            raise ValueError("余额不足，不能透支")
        self._balance -= valid_amount

    def __str__(self) -> str:
        return f"{self.owner} 的钱包余额：{self._balance} 分"

    def __repr__(self) -> str:
        return f"Wallet(owner={self.owner!r}, balance={self._balance})"

    def __eq__(self, other: object) -> bool:
        if not isinstance(other, Wallet):
            return NotImplemented
        return (self.owner, self._balance) == (other.owner, other._balance)

    @classmethod
    def from_text(cls, text: str) -> Self:
        """与 Account.from_text 同理：解析文本后通过 ``cls(...)`` 创建实际调用类。"""
        owner: str, separator: str, balance_text: str = text.partition(":")
        if not separator:
            raise ValueError("钱包文本必须使用“所有者:余额”格式")
        return cls(owner, int(balance_text.strip()))


class LimitedWallet(Wallet):
    """LimitedWallet 是一种 Wallet，并增加每日提现上限。"""

    def __init__(self, owner: str, balance: int = 0, daily_limit: int = 1_000) -> None:
```

```python

class LimitedWallet(Wallet):
    """LimitedWallet 是一种 Wallet，并增加每日提现上限。"""

    def __init__(self, owner: str, balance: int = 0, daily_limit: int = 1_000) -> None:
        # 先让 Wallet 建立 owner 和 _balance，再创建 LimitedWallet 自己新增的状态。
        super().__init__(owner, balance)
        self.daily_limit: int = self._validate_positive_amount(daily_limit)
        self._withdrawn_today = 0

    def withdraw(self, amount: int) -> None:
        """覆写父类取款：先增加每日限额规则，再复用父类的余额规则。"""
        valid_amount: int = self._validate_positive_amount(amount)
        if self._withdrawn_today + valid_amount > self.daily_limit:
            raise ValueError("超过每日提现上限")

        # 这里的 super().withdraw(...) 按 MRO 调用 Wallet.withdraw(...)。
        # 只有父类取款成功后，才累计今日已取金额；失败不会错误地占用每日额度。
        super().withdraw(valid_amount)
        self._withdrawn_today += valid_amount


class GiftCard:
    """GiftCard 拥有一个 Wallet，并维护自己的单笔消费上限。"""

    def __init__(self, wallet: Wallet, single_payment_limit: int) -> None:
        # 组合：保存传入的钱包引用。GiftCard 不是 Wallet 子类，而是把扣款委托给 Wallet。
        self.wallet: Wallet = wallet
        self.single_payment_limit: int = Wallet._validate_positive_amount(single_payment_limit)

    def pay(self, amount: int) -> None:
        """先检查礼品卡自己的单笔限额，再调用钱包的取款行为。"""
        valid_amount: int = Wallet._validate_positive_amount(amount)
        if valid_amount > self.single_payment_limit:
            raise ValueError("超过礼品卡单笔消费上限")
        self.wallet.withdraw(valid_amount)

```

```python
class BadWalletHistory:
    """教学反例：类属性交易记录会被所有实例共享。"""

    # 这个列表在 class 语句执行时只创建一次，不会在 BadWalletHistory() 时重新创建。
    # ClassVar 明确标出这是故意共享的类属性，运行时仍然只有这一个列表。
    transactions: ClassVar[list[str]] = []

    def record(self, message: str) -> None:
        # self 上找不到 transactions 时，会回到类上取得所有实例共享的那个列表。
        self.transactions.append(message)


class WalletHistory:
    """正确示例：每次初始化都创建独立的交易记录列表。"""

    def __init__(self) -> None:
        # 在 self 上创建列表，因此每个 WalletHistory 实例都有自己的列表。
        self.transactions: list[str] = []

    def record(self, message: str) -> None:
        self.transactions.append(message)


def demonstrate_creation_and_methods() -> None:
    show_section("8.1~8.4 对象创建与实例方法")

    # Python 不写 new。调用 Account 类对象后，Python 创建实例并调用 Account.__init__。
    account = Account(" Alice ", 10_000)
    print("实例类型:", type(account).__name__)
    print("初始化后的状态:", repr(account))
    print("户主已被规范化:", account.owner)

    # account.deposit 是绑定方法：Python 已经把 account 记作它的 self。
    bound_deposit: Callable[..., None] = account.deposit
    print("绑定方法记住当前实例:", bound_deposit.__self__ is account)
    bound_deposit(500)  # 等价于 Account.deposit(account, 500)
    print("调用绑定方法后:", account)
```

```
364 def demonstrate_creation_and_methods() -> None:
373     # account.deposit 是绑定方法：Python 已经把 account 记作它的 self。
374     bound_deposit: Callable[..., None] = account.deposit
375     print("绑定方法记住当前实例:", bound_deposit.__self__ is account)
376     bound_deposit(500)  # 等价于 Account.deposit(account, 500)
377     print("调用绑定方法后:", account)
378
379
380 def demonstrate_class_and_instance_attributes() -> None:
381     show_section("8.3 类属性与实例属性")
382
383     alice = Account("Alice")
384     bob = Account("Bob")
385     print("实例默认从类上读取:", alice.bank_name, bob.bank_name)
386
387     # 给 alice 同名赋值会创建实例属性，只遮蔽 alice 对类属性的读取。
388     alice.bank_name = "城东分行"
389     print("实例属性遮蔽类属性:", alice.bank_name)
390     print("其他实例仍读取类属性:", bob.bank_name)
391     print("类本身的值没有改变:", Account.bank_name)
392
393     first_bad_cart = BadShoppingCart()
394     second_bad_cart = BadShoppingCart()
395     first_bad_cart.items.append("Python 书")
396     print("反例中的两个购物车共享列表:", first_bad_cart.items is second_bad_cart.items)
397     print("第二个购物车被意外污染:", second_bad_cart.items)
398
399     first_cart = ShoppingCart()
400     second_cart = ShoppingCart()
401     first_cart.items.append("Python 书")
402     print("正确示例的列表彼此独立:", first_cart.items is not second_cart.items)
403     print("第二个购物车保持为空:", second_cart.items)
404
405
406 def demonstrate_property() -> None:
407     show_section("8.7~8.10 封装约定与 property")
```

```python
def demonstrate_property() -> None:
    show_section("8.7~8.10 封装约定与 property")

    account = Account("Alice", 1_000)
    print("类上的 balance 类型:", type(Account.balance).__name__)
    print("实例真正保存的属性:", vars(account))
    print("实例中没有第二份 balance:", "balance" not in vars(account))
    print("读取 property:", account.balance)

    account.balance = 800  # 看似普通赋值，实际会调用 balance setter。
    print("setter 写入后的余额:", account.balance)

    try:
        account.balance = -1
    except ValueError as error:
        print("负数被 setter 拒绝:", error)

    try:
        Account("Bob", "100")  # type: ignore[arg-type]  # 教学反例：类型提示不做运行时转换
    except TypeError as error:
        print("字符串余额被运行时校验拒绝:", error)

    print("True 虽是 int 子类，但不是合法金额:", Account.is_valid_amount(True))


def demonstrate_protocols() -> None:
    show_section("8.5 Python 数据模型与魔术方法")

    first = Account("Alice", 1_500)
    same_value = Account("Alice", 1_500)
    print("str 面向用户:", str(first))
    print("repr 面向开发者:", repr(first))
    print("== 比较领域值:", first == same_value)
    print("is 比较对象身份:", first is same_value)
    print("与不支持的类型比较:", first == "Alice:1500")
    print("可变值对象默认不可哈希:", Account.__hash__ is None)
```

```python
def demonstrate_constructors_and_inheritance() -> None:
    show_section("8.6 与 8.11 继承、类方法和静态方法")

    # 调用类方法时不手动传 cls；Python 会把点号左边的实际类自动绑定进去。
    account: Account = Account.from_text("Alice:10000")
    savings: SavingsAccount = SavingsAccount.from_text("Bob:20000")
    print("Account.from_text 创建:", repr(account))
    print("cls 保留实际调用的子类:", type(savings).__name__)

    # 绑定后的 classmethod 会通过 __self__ 暴露它记住的类，用来观察机制即可。
    print("Account.from_text 绑定的 cls:", Account.from_text.__self__.__name__)
    print("子类 from_text 绑定的 cls:", SavingsAccount.from_text.__self__.__name__)

    # 静态方法没有自动绑定的 self 或 cls，调用形式与 Java 静态方法最接近。
    print("静态方法校验金额:", Account.is_valid_amount(100))
    print("SavingsAccount 的 MRO:", [item.__name__ for item in SavingsAccount.mro()])

    assert isinstance(savings, SavingsAccount)
    savings.interest_rate = 0.05
    added: int = savings.add_interest()
    print("super 初始化的子类:", repr(savings))
    print("本次增加的利息:", added)

    # 子类对象可以出现在需要 Account 的位置；循环只依赖共同的公开接口。
    accounts: list[Account] = [account, savings]
    for current in accounts:
        print("多态处理:", current)


def demonstrate_composition() -> None:
    show_section("组合优先于勉强继承")

    account = Account("Cara", 8_000)
    card = BankCard(account, single_payment_limit=3_000)
    card.pay(2_500)
    print("银行卡委托账户扣款:", account.balance)

```

```
473 def demonstrate_composition() -> None:
481     try:
482         card.pay(3_500)
483     except ValueError as error:
484         print("组合对象维护自己的限制:", error)
485     print("失败后余额没有变化:", account.balance)
486
487
488 def demonstrate_readme_exercises() -> None:
489     """运行 README 中 Wallet、继承、组合与共享状态练习。"""
490     show_section("README 小练习")
491
492     print("练习 1: Wallet 基础对象与不变量")
493     empty_wallet = Wallet("Alice")
494     print("  零余额:", str(empty_wallet), "/", repr(empty_wallet))
495     print("  Wallet.balance 没有 setter:", Wallet.balance.fset is None)
496
497     invalid_balances: tuple[tuple[str, object], ...] = (
498         ("负数", -1),
499         ("字符串", "100"),
500         ("布尔值", True),
501     )
502     for label, balance in invalid_balances:
503         try:
504             Wallet("Bad", balance)  # type: ignore[arg-type]
505         except (TypeError, ValueError) as error:
506             print(f"  {label}余额被拒绝:", error)
507
508     exact_wallet = Wallet("Bob", 500)
509     exact_wallet.withdraw(500)
510     print("  刚好取完:", exact_wallet.balance)
511     try:
512         exact_wallet.withdraw(1)
513     except ValueError as error:
514         print("  透支被拒绝:", error)
515     try:
```

```
488  def demonstrate_readme_exercises() -> None:
515      try:
516          exact_wallet.deposit(0)
517      except ValueError as error:
518          print("  非正数存款被拒绝:", error)
519
520      print("练习 2: 替代构造器与相等")
521      first: Wallet = Wallet.from_text("Alice:100")
522      second = Wallet("Alice", 100)
523      print("  from_text 结果:", repr(first))
524      print("  值相等/身份不同:", first == second, first is not second)
525      print("  与不支持类型比较:", first == "Alice:100")
526      try:
527          Wallet.from_text("Alice 100")
528      except ValueError as error:
529          print("  缺少冒号:", error)
530
531      print("练习 3: 继承与组合")
532      limited = LimitedWallet("Cara", 2_000, daily_limit=1_000)
533      limited.withdraw(600)
534      try:
535          limited.withdraw(500)
536      except ValueError as error:
537          print("  每日限额:", error)
538      print("  限额钱包余额:", limited.balance)
539
540      gift_wallet = Wallet("Dan", 2_000)
541      gift_card = GiftCard(gift_wallet, single_payment_limit=500)
542      gift_card.pay(300)
543      try:
544          gift_card.pay(600)
545      except ValueError as error:
546          print("  单笔限额:", error)
547      print("  礼品卡委托钱包后的余额:", gift_wallet.balance)
548      print("  关系: LimitedWallet 是一种 Wallet; GiftCard 拥有一个 Wallet")
549
550      print("练习 4: 定位共享状态 bug")
```

```
488 def demonstrate_readme_exercises() -> None:
549
550     print("练习 4: 定位共享状态 bug")
551     BadWalletHistory.transactions.clear()  # 重跑示例前先清理类级教学状态。
552     bad_first = BadWalletHistory()
553     bad_second = BadWalletHistory()
554     bad_first.record("存款 100 分")
555     print("  错误版本第二个实例也有记录:", bad_second.transactions)
556     print("  错误版本共享同一列表:", bad_first.transactions is bad_second.transactions)
557
558     good_first = WalletHistory()
559     good_second = WalletHistory()
560     good_first.record("存款 100 分")
561     print("  修复后第二个实例为空:", good_second.transactions)
562     print("  修复后列表相互独立:", good_first.transactions is not good_second.transactions)
563
564
565 def main() -> None:
566     demonstrate_creation_and_methods()
567     demonstrate_class_and_instance_attributes()
568     demonstrate_property()
569     demonstrate_protocols()
570     demonstrate_constructors_and_inheritance()
571     demonstrate_composition()
572     demonstrate_readme_exercises()
573
574
575 if __name__ == "__main__":
576     main()
577
```